Produzindo um projeto (Parte 1)

Feito por JJMAGame / Jorge

Commit inicial

Criei um "hello world" para iniciar algumas linhas de programação, minha prioridade inicial é criar um sistema de login que fale com o código json, mas por enquanto eu to fazendo a linha de código

Código criado:

Hello-World

```java
public class main {
    public static void main(String[] args) {
        System.out.println("Hello, World");
    }
}
```

Fazendo as primeira partes do código

Fazendo um sistema de usuário usando POO, para conversa com o json, por hora fiz o sistema this.nome as **10:52:23** estarei alogando mais o código cada fez que for evoluindo

Encapsulamento

```java
public class usuario {
    private String nome;
    private String email;
    private String senha;
    public usuario(String nome, String email, String senha) {
        this.nome = nome;
        this.email = email;
        this.senha = senha;
    }
}
```

Após a construção dos **Strings** foi incluindo também o **Getter e Setter** do usuario

Getter-and-Setter

```
`//getter`

    public String getNome() {
        return nome;
    }

    public String getEmail() {
        return email;
    }

    public String getSenha() {
        return senha;
    }



    //setter



    public void setNome(String nome) {
        this.nome = nome;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public void setSenha(String senha) {
        this.senha = senha;
    }
```

Melhorando o Obsidian para ter integração com o Git

Colocando o a validação de login no Usuário para criação e validação de conta



Validação-no-Setter

```
    // ===== SETTERS COM VALIDAÇÃO =====

    public void setNome(String nome) {
        if (nome == null || nome.trim().isEmpty()) {
            throw new IllegalArgumentException("Nome não pode ser
vazio");
        }
        this.nome = nome.trim();
```

```
8       }
9
10      public void setEmail(String email) {
11          if (email == null || !email.contains("@")) {
12              throw new IllegalArgumentException("Email inválido");
13          }
14          this.email = email.toLowerCase().trim();
15      }
16
17
18
19      public void setSenha(String senha) {
20          if (senha == null || senha.length() < 8) {
21              throw new IllegalArgumentException("Senha deve ter no mínimo
    8 caracteres");
22
23          }
24          this.senha = senha;
25      }
26
27
28
29      // ===== MÉTODO DE NEGÓCIO (reaproveitamento futuro) =====
30
31      public boolean autenticar(String email, String senha) {
32          return this.email.equalsIgnoreCase(email) &&
    this.senha.equals(senha);
33
34      }
35
36      @Override
37      public String toString() {
38          return "Usuario{nome='" + nome + "', email='" + email + "'}";
39      }
40  }
41
```

Ainda tenho que validar se estar tudo certo se esta no caminho certo para o encapsulamento e herança, terei que revisar o Pilares. *Quais seria os pilares?*

Abstração:

Consiste em focar nos aspectos essenciais de um objeto do mundo real e representá-los no código por meio de atributos (características) e métodos (ações), ignorando detalhes complexos e desnecessários.

Encapsulamento:

É a prática de esconder os detalhes internos de funcionamento de um objeto e proteger seus dados (atributos), controlando o acesso a eles por meio de métodos específicos (como *getters* e *setters*)

Herança:

Permite que uma classe (subclasse) receba ou herde atributos e comportamentos de outra classe (superclasse), facilitando a reutilização de código e a criação de hierarquias.

Polimorfismo:

É a capacidade de objetos de diferentes classes responderem à mesma mensagem ou método de maneiras diferentes, adaptando o comportamento conforme o contexto de cada subclasse.

O Encapsulamento esta sendo construído 15/10/2026

Foi incluido todo o sistema validação dentro do código e ele esta interagindo no main mas o main ainda vai ser o ponto de entrada, vou dividir as 2 telas tanto do inquilino, tanto do administrador

Teste-do-código

```java
public class main {

    public static void main(String[] args) {
        Usuario u = new Usuario("Jorge", "jjma@hotmail.com", "123456");

        System.out.println(u);
        System.out.print(u.getEmail());

        // Teste de Validação

        try {
            u.setEmail("invalido");
        } catch (IllegalArgumentException e) {
            System.out.println("Erro: " + e.getMessage());
        }

        System.out.println(u.autenticar("jjma@hotmail.com", "123456")); // True
        System.out.println(u.autenticar("jjma@hotmail.com", "error")); // False
    }
}
```

Nota de atualização para o uso da aplicação para visualizar se um dos pilares foi incluso

---

Colocando o sistema Gson no código

O maior desafio é incluir um sistema json dentro do código usando o sistema e depedencia gson

JsonUtil

```java
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import com.google.gson.reflect.TypeToken;
import java.io.*;
import java.lang.reflect.Type;
import java.util.ArrayList;
import java.util.List;

public class JsonUtil {

    private static final String ARQUIVO = "usuarios.json";

    private static final Gson gson = new GsonBuilder().setPrettyPrinting().create();

    // Lê a lista de usuários do arquivo JSON

    public static List<Usuario> lerUsuarios() {
        File file = new File(ARQUIVO);
        if (!file.exists()) return new ArrayList<>();



        try (Reader reader = new FileReader(file)) {
            Type tipoLista = new TypeToken<List<Usuario>>() {}.getType();
            List<Usuario> usuarios = gson.fromJson(reader, tipoLista);
            return usuarios != null ? usuarios : new ArrayList<>();

        } catch (IOException e) {
            throw new RuntimeException("Erro ao ler o arquivo JSON", e);

        }
    }
}
```

Problemas:

mesmo que o sistema gson.jar tenha sido instalado corretamente, começou a ter alguns problemas incomuns então estou tentando localizar o problema para ter a efetividade do programa, tentei colocar o pom.xml, mas não sei realmente se foi efetivo para resolver o problema

pom

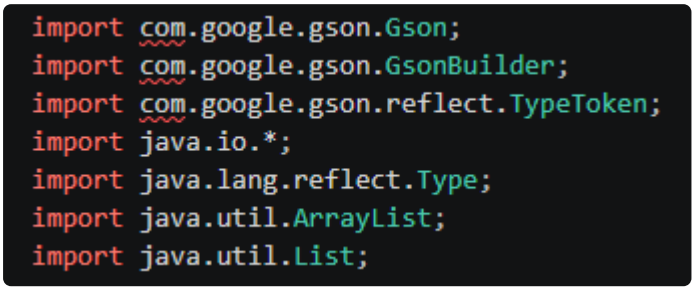
```
<dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.10.1</version>
</dependency>
```

O que foi notado que o sistema no settings.json realmente funcionou mas o sistema não esta detectando o "com." coisa que eu tenho que ver futuramente.

settings

```
{
    "java.project.referencedLibraries": [
        "D:\\GITHUB REPOSITORY\\Nexus-painel-menu\\lib\\gson-2.10.1.jar",
        "lib/**/*.jar",
    ]
}
```

Imagem que explicar melhor o problema



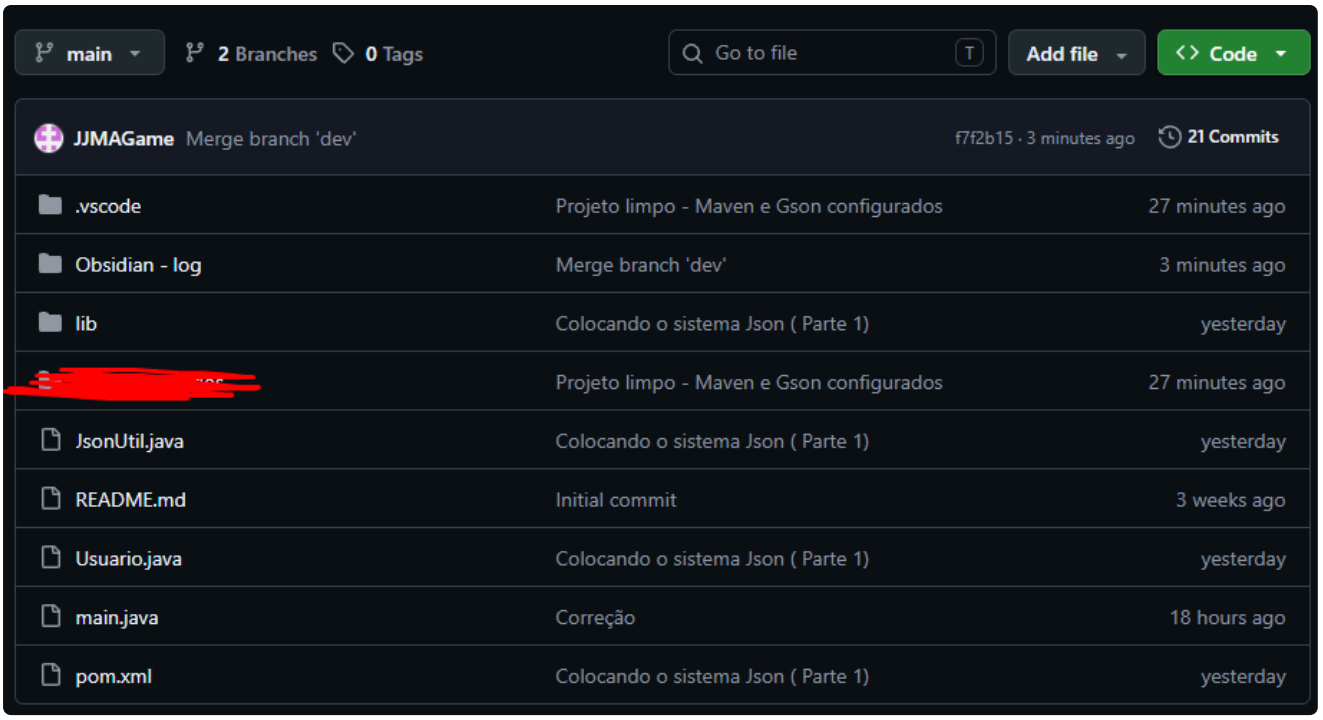
O sistema esta detectando bem o todo o resto do código mas não pega o com. irei ver isso futuramente e atualizarei esse documento

---

Parte 2

No momento que eu estou colocando esse texto estou me arrumando para a faculdade, conseguir construir um sistema Maven e junto com a dependencia Gson, agora eu consigo montar um banco de dados localmente

Antes



Depois



Todo-o-sistema-pom

```
<?xml version="1.0" encoding="UTF-8"?>



<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId>com.nexusmaven</groupId>
  <artifactId>sistema-usuarios</artifactId>
  <version>1.0-SNAPSHOT</version>
  <name>sistema-usuarios</name>
```

```
15    <!-- FIXME change it to the project's website -->
16    <url>http://www.example.com</url>
17    <properties>
18      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
19      <maven.compiler.source>1.8</maven.compiler.source>
20      <maven.compiler.target>1.8</maven.compiler.target>
21    </properties>
22    <dependencies>
23      <dependency>
24        <groupId>junit</groupId>
25        <artifactId>junit</artifactId>
26        <version>4.11</version>
27        <scope>test</scope>
28      </dependency>
29      <dependency>
30        <groupId>com.google.code.gson</groupId>
31        <artifactId>gson</artifactId>
32        <version>2.10.1</version>
33      </dependency>
34    </dependencies>
35
36    <build>
37      <pluginManagement><!-- lock down plugins versions to avoid using
    Maven defaults (may be moved to parent pom) -->
38        <plugins>
39          <!-- clean lifecycle, see
    https://maven.apache.org/ref/current/maven-
    core/lifecycles.html#clean_Lifecycle -->
40          <plugin>
41            <artifactId>maven-clean-plugin</artifactId>
42            <version>3.1.0</version>
43          </plugin>
44          <!-- default lifecycle, jar packaging: see
    https://maven.apache.org/ref/current/maven-core/default-
    bindings.html#Plugin_bindings_for_jar_packaging -->
45          <plugin>
46           <artifactId>maven-resources-plugin</artifactId>
47            <version>3.0.2</version>
48          </plugin>
49          <plugin>
50            <artifactId>maven-compiler-plugin</artifactId>
51            <version>3.8.0</version>
52          </plugin>
53          <plugin>
54            <artifactId>maven-surefire-plugin</artifactId>
55            <version>2.22.1</version>
56          </plugin>
57          <plugin>
58            <artifactId>maven-jar-plugin</artifactId>
```

```
            <version>3.0.2</version>
          </plugin>
          <plugin>
            <artifactId>maven-install-plugin</artifactId>
            <version>2.5.2</version>
          </plugin>
          <plugin>
            <artifactId>maven-deploy-plugin</artifactId>
            <version>2.8.2</version>
          </plugin>
          <!-- site lifecycle, see
    https://maven.apache.org/ref/current/maven-
    core/lifecycles.html#site_Lifecycle -->
          <plugin>
            <artifactId>maven-site-plugin</artifactId>
            <version>3.7.1</version>
          </plugin>
          <plugin>
            <artifactId>maven-project-info-reports-plugin</artifactId>
            <version>3.0.0</version>
          </plugin>
        </plugins>
      </pluginManagement>
    </build>
</project>
```

Parte 3

Funcionou, conseguir criar uma commit limpa e colocar um **Apache Maven** utilizando no Visual Studio e todos os códigos foram recorrigidos e atributos adicionados:

JsonUtil-Novo

```java
package com.nexusmaven;



import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.io.Reader;
import java.io.Writer;
import java.lang.reflect.Type;
import java.util.ArrayList;
import java.util.List;
```

```java


import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import com.google.gson.reflect.TypeToken;



public class JsonUtil {

    private static final String ARQUIVO = "usuarios.json";
    private static final Gson gson = new
GsonBuilder().setPrettyPrinting().create();



    // Lê a lista de usuários do arquivo JSON

    public static List<Usuario> lerUsuarios() {
        File file = new File(ARQUIVO);
        if (!file.exists()) return new ArrayList<>();



        try (Reader reader = new FileReader(file)) {
            Type tipoLista = new TypeToken<List<Usuario>>() {}.getType();
            List<Usuario> usuarios = gson.fromJson(reader, tipoLista);
            return usuarios != null ? usuarios : new ArrayList<>();
        } catch (IOException e) {
            throw new RuntimeException("Erro ao ler o arquivo JSON", e);
        }
    }



    public static void salvarUsuarios(List<Usuario> usuarios) {
        try (Writer writer = new FileWriter(ARQUIVO)) {
            gson.toJson(usuarios, writer);
        } catch (IOException e) {
            throw new RuntimeException("Erro ao salvar o arquivo JSON",
e);

        }

    }

}
```

Main-Restruturado

```java
package com.nexusmaven;



import java.util.ArrayList;
import java.util.List;

public class main {

    public static void main(String[] args) {

        // 1. Criando usuários (Senha com 8 caracteres!)
        Usuario u1 = new Usuario("Jorge", "jjma@hotmail.com", "12345678");
        Usuario u2 = new Usuario("admin", "admin@hotmail.com", "admin123");

        // 2. Colocando em uma lista

        List<Usuario> listaUsuarios = new ArrayList<>();
        listaUsuarios.add(u1);
        listaUsuarios.add(u2);

        // 3. Salvando usuários no arquivo JSON

        JsonUtil.salvarUsuarios(listaUsuarios);
        System.out.println("Usuários salvos no arquivo JSON.");


        // 4. Lendo do arquivo JSON e validar o save

        System.out.println("Lendo usuários do arquivo JSON:");
        List<Usuario> usuariosLidos = JsonUtil.lerUsuarios();

        for (Usuario u : usuariosLidos) {
            System.out.println(u);

        }

        // 5. Testando autenticação

        if (!usuariosLidos.isEmpty()) {
            Usuario teste = usuariosLidos.get(0);
            boolean logado = teste.autenticar("jjma@hotmail.com", "12345678");
            System.out.println("\n Teste de login (Jorge): " + (logado ? "Autenticado com sucesso!" : "Falha na autenticação!"));
```

```
44
45          }
46      }
47  }
```



Usuario-remodelado

```
1   package com.nexusmaven;
2
3
4
5   public class Usuario {
6
7       private String nome;
8       private String email;
9       private String senha;
10
11      // Construtor reaproveita os setters (validação garantida)
12
13      public Usuario(String nome, String email, String senha) {
14
15          setNome(nome);
16          setEmail(email);
17          setSenha(senha);
18
19      }
20
21
22
23      // ===== GETTERS =====
24
25      public String getNome() { return nome; }
26      public String getEmail() { return email; }
27      public String getSenha() { return senha; }
28
29
30
31      // O Gson precisa do getter para salvar a senha no arquivo
32
33      public void setNome(String nome) {
34          if (nome == null || nome.trim().isEmpty()) {
35              throw new IllegalArgumentException("Nome não pode ser
    vazio");
36
37          }
38
39          this.nome = nome.trim();
40
```

```
    }


    public void setEmail(String email) {
        if (email == null || email.trim().isEmpty()) {
            throw new IllegalArgumentException("Email não pode ser vazio");

        }

        this.email = email.trim();

    }



    public void setSenha(String senha) {
        if (senha == null || senha.length() < 8) {
            throw new IllegalArgumentException("Senha deve ter pelo menos 8 caracteres");

        }

        this.senha = senha;

    }



    public boolean autenticar(String email, String senha) {
        return this.email.equalsIgnoreCase(email) && this.senha.equals(senha);

    }



    @Override
    public String toString() {
        return "Usuario{nome='" + nome + "', email='" + email + "'}";

    }

}
```

No processo de teste finalmente o código conseguiu criar um json com os dados de login do cliente, mas ainda falta as outras informações mas terei que formular primeiro todo o sistema de login e senha utilizando o scan mas é um processo

Arquivo-de-usuario-criado-pelo-programa

```
[

  {
    "nome": "Jorge",
    "email": "jjma@hotmail.com",
    "senha": "12345678"
  },

  {
    "nome": "admin",
    "email": "admin@hotmail.com",
    "senha": "admin123"
  }

]
```

O que foi descoberto nesse meio tempo

Que a maioria dos programas de IDE um dos exemplos é o eclipse e IJ usam um sistema de arquitetura chamada Maven, aqui uma preve explicação



O Maven é uma ferramenta de automação de build e gerenciamento de projetos para Java. Em termos simples, ele padroniza e automatiza tarefas repetitivas como compilar código, rodar testes, empacotar o programa e baixar bibliotecas de terceiros.

🛠️ *Como funciona sem Maven?*

O próprio JDK (Java Development Kit) já traz as ferramentas básicas para trabalhar sem ferramentas externas de build:

- ***Compilação e Execução**: Você pode usar o comando `javac` para compilar seus arquivos `.java` e o comando `java` para executar o bytecode gerado .*
- ***Gerenciamento Manual**: Sem o Maven, você precisa baixar manualmente os arquivos `.jar` (bibliotecas de terceiros) e configurá-los no classpath da sua aplicação para que o Java os encontre durante a compilação e execução .*

- ***Outras Ferramentas**: Existem alternativas ao Maven, como o **Gradle** (muito popular no desenvolvimento Android) ou até mesmo o **Ant** (mais antigo, mas ainda usado) .*

📦 *Por que o Maven é tão usado então?*

O Maven resolve problemas complexos que o `javac` puro não cobre, principalmente em projetos grandes:

1. ***Gerenciamento de Dependências**: Em vez de baixar dezenas de `.jar` manualmente, você declara a dependência no arquivo `pom.xml` . O Maven baixa a biblioteca e todas as suas dependências transitivas automaticamente .*
2. ***Ciclo de Vida Padronizado**: Ele define passos claros (`clean` , `compile` , `test` , `package`) que garantem que qualquer pessoa na equipe consiga compilar o projeto da mesma forma, evitando o clássico problema de "na minha máquina funciona" .*
3. ***Estrutura de Projeto**: O Maven impõe uma estrutura de pastas padrão (ex: `src/main/java`), o que facilita a compreensão do projeto por novos desenvolvedores .*

Pesquisa feita pelo DeepSeek

Por agora estarei fazendo um doc dos processos para depois ser utilizado na apresentação tecnica

---

Fim

Essa é a primeira versão de um documento para a parte técnica e validação de Springs então mas documentos serão desenvolvido ao avançar do projeto

Nexus